Chenxi Ma

LLM Inference Systems / Machine Learning Systems Engineer

Beijing, China | +86 15387087330 | shannon20051127@gmail.com | github.com/algebra-MCX

Education

Wuhan University of Technology — Expected June 2028
B.Eng. in Mechanical Engineering — Wuhan, China

Open-Source Engineering

vLLM Project — 2026 – Present
Open-Source Contributor – Model Runner V2 / DBO / MoE — GitHub

I. MRV2 DBO Execution Correctness and Failure Handling

- **Routed-Expert Indexing Fix (Merged):** #63372 (Merged). Fixed incorrect routed-expert writes in the MRV2 Dual-Batch Overlap (DBO) execution path by indexing each microbatch using its corresponding token offset. In a 32-request DeepSeek-V2-Lite test, expert-routing agreement improved from 0% on the main branch to 89.4% with DBO enabled.
- **Microbatch Exception Propagation (Merged):** #63476 (Merged). Improved exception handling in `UBatchRunner.finish()` to report the first recorded microbatch failure, preserving the original root cause instead of allowing it to be obscured by subsequent exceptions. Added GPU regression tests with 2/2 passing on H200.
- **DBO Worker Failure Recovery (Draft):** #64718 (Draft). Designed a failure-propagation and worker-unwinding mechanism to prevent sibling workers from remaining blocked on synchronization primitives after a microbatch fails. The proposed changes preserve the original exception and prevent indefinite hangs in `UBatchRunner.finish()`. Integration is planned after the MRV2–MRV1 dependency-removal work in #64668 (Draft).

II. CUDA Graph Execution and Inference Performance

- **MLA Decode Staging (Open):** #63556 (Open). Proposed extending MLA decode staging to support captured FULL microbatch CUDA Graphs under imbalanced data-parallel workloads, targeting more efficient CUDA Graph replay in MRV2 DBO execution. The proposal received maintainer attention and prompted discussion of an alternative design; the final implementation approach remains under review.
- **Performance Evaluation:** In the documented 63/97-load A/B benchmark, the proposed implementation improved output throughput by 15.52% and reduced p95 Time Per Output Token (TPOT) from 132.017 ms to 113.930 ms, a reduction of approximately 13.7%. Evaluated execution strategies using GPU benchmarks focused on throughput and tail latency.

III. MoE Communication Correctness – NIXL / DeepEP

- **NIXL Expert Mapping (Open):** #64086 (Open). Fixed invalid expert-ID handling during global-to-physical expert remapping, preserving the `-1` padding sentinel instead of incorrectly converting invalid entries into valid physical expert indices. Received positive technical feedback from a Red Hat engineer reviewing the changes; awaiting the `ready` label.
- **DeepEP Low-Latency Expert Mapping (Open):** #64340 (Open). Fixed sentinel propagation in round-robin expert remapping across DeepEP Low-Latency dispatch and combine paths. Consolidated the relevant mapping logic and extended shared regression coverage. Received positive technical feedback from a Red Hat engineer reviewing the changes; awaiting the `ready` label.
- **Correctness Validation:** Developed CPU adapter regression tests covering mixed valid/invalid expert IDs, all-padding inputs, and linear expert placement. All 6 regression cases passed across the NIXL and DeepEP LL backends.

IV. MRV2 DBO Architecture Refactoring

- **Removing MRV1 Dependencies (Draft):** #64668 (Draft). Refactored MRV2 DBO microbatch slicing to compute padded slices directly within MRV2, eliminating its dependency on MRV1's `maybe_create_ubatch_slices` implementation. This change contributes to decoupling the MRV2 execution path from legacy MRV1 components.
- Updated MRV2 microbatch slicing tests with explicit expected outputs for five batch layouts, removing reliance on MRV1 implementations as the test oracle and improving the independence and maintainability of MRV2 DBO regression tests.

Research Experience

University of Washington – Systems Lab — May 2026 – Present
Research Intern — Seattle, WA / Remote

- Third author of *TraceLab: Characterizing Coding Agent Workloads for LLM Serving* (arXiv:2606.30560), investigating coding-agent execution patterns and their implications for large language model serving systems.
- Contributed to the analysis of approximately 4,300 coding-agent sessions, 350,000 LLM inference steps, and 430,000 tool calls, characterizing request distributions, context lengths, and tool-interaction behavior in agentic workloads.

- Analyzed long-running autonomous execution loops, long-input/short-output patterns, heavy-tailed tool-call distributions, and prefix-cache hit rates, providing empirical insights for performance modeling and optimization of agent-oriented LLM serving systems.

Technical Skills

LLM Inference Systems: vLLM, LLM Serving, MRV2, Dual-Batch Overlap, Microbatch Execution, CUDA Graphs, Prefill / Decode

Distributed Inference and MoE: Data Parallelism (DP), Expert Parallelism (EP), NIXL, DeepEP, Expert Routing, Dispatch / Combine

Performance and Correctness: GPU Profiling, Benchmarking, Throughput / TPOT, CUDA Graph Replay, Regression Testing, Failure Propagation

Engineering Tools: Python, PyTorch, pytest, Git / GitHub, Linux, NVIDIA H200 / B200